Saqif Kaisar

Department of CSE, UIU

CSE 3421 – Software Engineering

The Big Idea

Design patterns are one of the most fundamental concepts in object-oriented software development.

Instead of solving every software design problem from scratch, we can reuse proven solutions to common problems.

Problem → Proven Solution → Reusable Design

Introduction to Design Patterns – What is a Software Design Pattern?

A software design pattern is:

- A standard solution to a common programming problem.
- A description or template that can be reused in many different situations.
- A technique for making code more flexible or efficient.
- A way to ensure loose coupling in code.
- A way to increase the understandability of a program.

Remember

A design pattern is **not a ready-made piece of code**. It is a reusable **design idea**.

The famous book on design patterns was written by four authors, commonly known as the:

Gang of Four (GoF)

They described **23 design patterns**.

These patterns are divided into three categories:

1. **Creational**
2. **Structural**
3. **Behavioral**

Gang of Four – Three Categories

Creational
How objects are instantiated.

Structural
How objects/classes can be combined.

Behavioral
How objects communicate.

Creational	Structural	Behavioral
5 patterns	7 patterns	11 patterns

Creational Patterns – Overview

Purpose

Creational patterns deal with **how objects are instantiated**.

There are five creational patterns:

1. Factory Method
2. Abstract Factory
3. Singleton
4. Builder
5. Prototype

Structural Patterns – Overview

Purpose

Structural patterns describe how objects and classes can be **combined**.

There are seven structural patterns:

- Adapter
- Bridge
- Composite
- Decorator
- Façade
- Flyweight
- Proxy

Behavioral Patterns – Overview

Purpose

Behavioral patterns describe **how objects communicate**.

- Interpreter
- Template Method
- Chain of Responsibility
- Command
- Iterator
- Mediator
- Memento
- Observer
- State
- Strategy
- Visitor

UML – Important Relationships

Some important UML relationships:

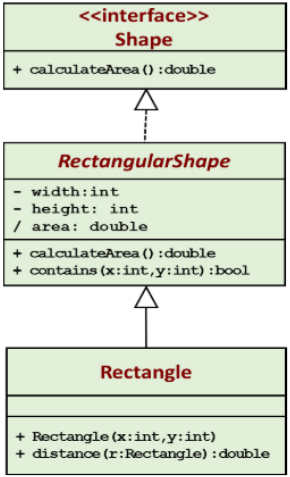
Generalization: Represents an inheritance relationship.

Dependency: "Uses"

Composition: "Is entirely made of"

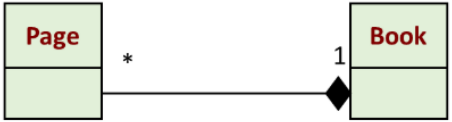
Aggregation: "Is part of"

UML – Generalization, Dependency, Composition & Aggregation





Aggregation: “is part of”



Composition: “is entirely made of”



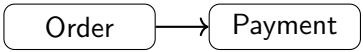
Coupling & Cohesion – Key Principles

Coupling

The degree to which one module depends on another module.

Goal: Low Coupling

Example:



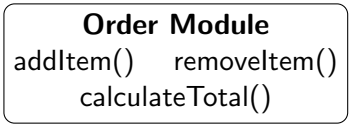
Order depends on Payment.

Cohesion

The degree to which the elements within a module belong together.

Goal: High Cohesion

Example:



All functions are related to orders.

Good Design

Low Coupling + High Cohesion

Factory Method

The Factory Method is a **creational design pattern**.

Main Idea

Instead of creating objects directly, we delegate object creation to a **factory**.

Factory Method – Definition

The Factory Method:

- Defines an interface for creating related classes.
- A factory class takes responsibility for object instantiation through a factory method.
- The client gets a class instance using that factory method.

Client → Factory → Concrete Object

Factory Method – Why Use a Factory?

Suppose a program needs different types of shapes.

Without a factory:

- Client directly creates `Circle`.
- Client directly creates `Rectangle`.
- Client directly creates `Square`.

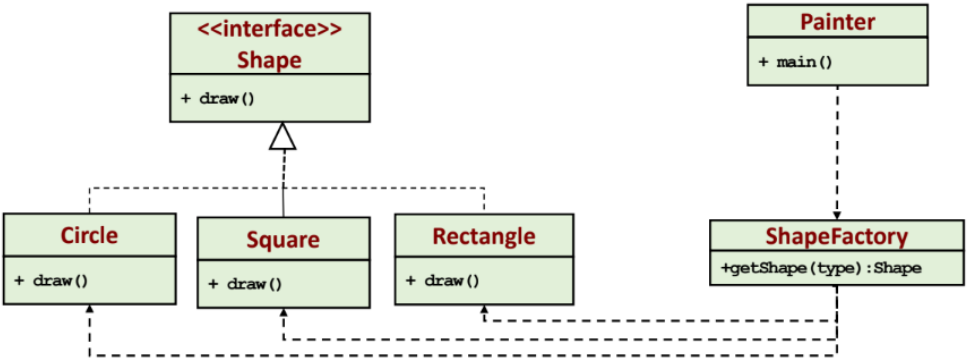
Problem

The client becomes tightly coupled with the concrete classes.

Solution

Let a **ShapeFactory** create the appropriate object.

Factory Method – Shape Example



Factory Method – How the Factory Works

Suppose the client requests:

```
getShape("Circle")
```

The process is:

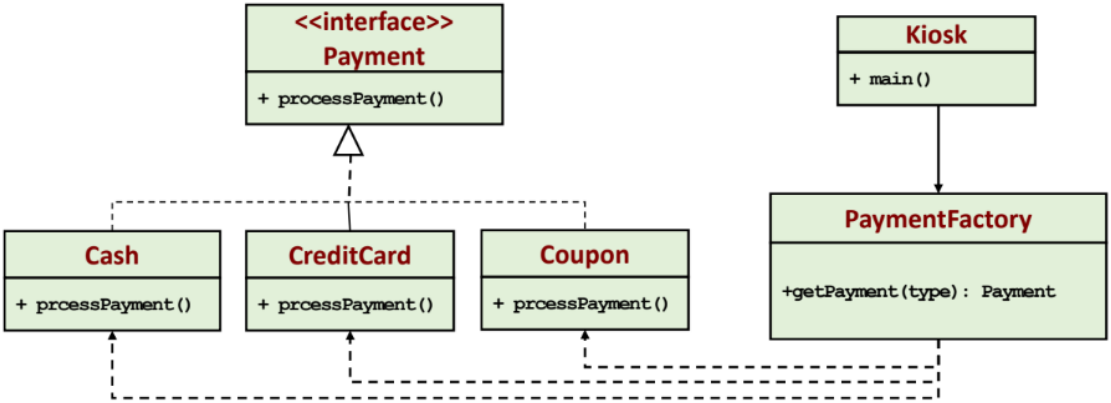
1. Client asks the factory for a shape.
2. Factory checks the requested type.
3. Factory creates the appropriate concrete object.
4. Factory returns the object through the `Shape` interface.

$$\text{Client} \rightarrow \text{Factory} \rightarrow \text{Circle}$$

Factory Method – Kiosk Example

Consider a kiosk that accepts different payment methods:

- Cash or Coupon or Credit Card



Factory Method – Consequences

The Factory Pattern:

- Removes instantiation of actual implementation classes from client code.
- Makes code more robust.
- Reduces coupling.
- Makes the system easier to extend.
- Allows implementation classes to change without affecting the client.
- Provides abstraction between implementation and client classes through inheritance.

Factory Method – Practice Problems

Problem 1: Travel Site

A travel site allows users to book:

- Bus tickets
- Train tickets
- Flight tickets

The user provides the travel type as:
`bus`, `train`, or `flight`.

Problem 2: Garment Factory

A factory produces:

- Shirt
- Pant
- Trousers

Task: Draw UML diagrams using the appropriate design pattern.

Abstract Factory

Main Idea

The Abstract Factory Pattern provides an encapsulation mechanism to a **group of factories**.

Each factory can be accessed through an **Abstract Factory** class.

Abstract Factory – Definition

The Abstract Factory:

- Encapsulates a group of factories.
- Provides access to individual factories through an abstract factory class.
- Uses the Factory Pattern as a sub-concept.
- Each concrete factory follows the Factory Pattern.

Factory of Factories

Factory Method vs Abstract Factory

Factory Method

Creates an object from a family/type.

Client $\longrightarrow$ Factory $\longrightarrow$ Product

Abstract Factory

Creates related families of objects.

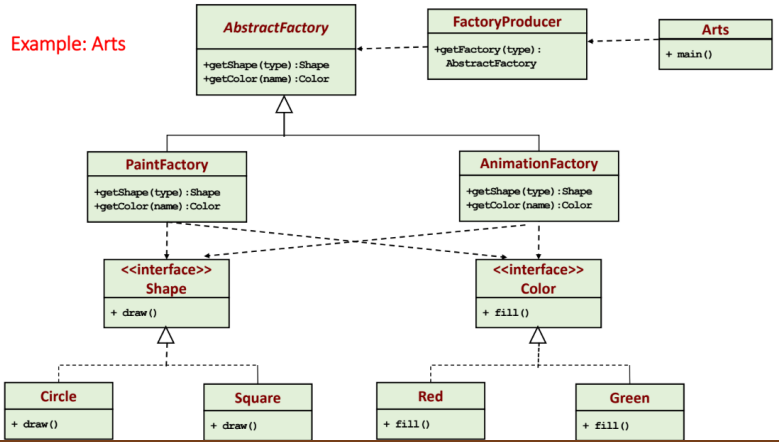
Client $\longrightarrow$ AbstractFactory $\longrightarrow$ Factory $\longrightarrow$ Products

Abstract Factory – Arts Example

Suppose an application can work with:

- Different shapes or Different Shapes

There can be multiple families of products.

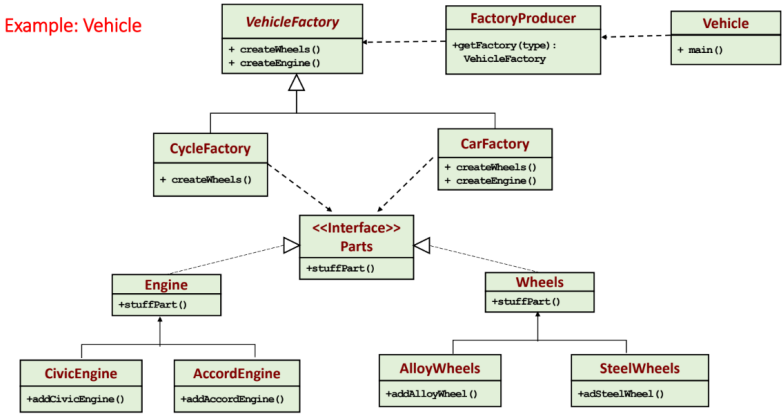


Abstract Factory – Vehicle Example

A vehicle may require a compatible:

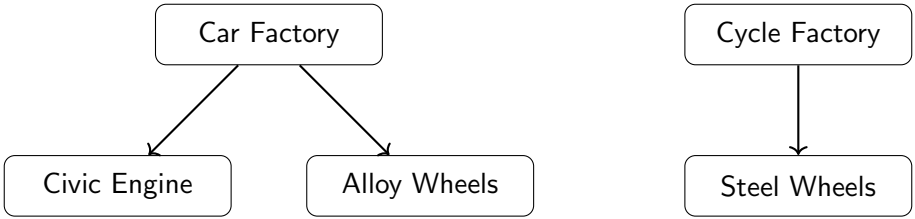
- Engine or Wheels

Different factories can produce different vehicle families.



Abstract Factory – Product Families

Example of compatible products:



Important

The Abstract Factory helps maintain **consistency among related products**.

Abstract Factory – Consequences

The Abstract Factory Pattern:

- Acts as a "factory of factories".
- Can be extended to accommodate more products.
- Isolates concrete classes from the client.
- Promotes consistency among products.

Abstract Factory – Practice: Coffee Shop

A coffee shop serves:

- Tea
- Coffee

Both require ingredients such as:

- Coffee powder
- Tea leaves
- Sugar

Task:
Draw a UML diagram using the appropriate design pattern.

Abstract Factory – Practice: Microwave-Safe Utensils

There are two groups of utensils:

- Microwave-safe products
- Non-microwave-safe products

If microwave-safe products are required, the system should provide compatible:

- Bowl
- Plate
- Cup

Constraint

Microwave-safe and non-microwave-safe products should not be mixed.

Task:

Draw a UML diagram using the appropriate design pattern.

Singleton

Definition

Ensure that a class has **only one instance** and provide a **global point of access** to it.

The Singleton pattern also facilitates:

Lazy Initialization

Singleton – Why Singleton?

Suppose an application needs a database connection.

Do we want every part of the application to create a new connection?

No!

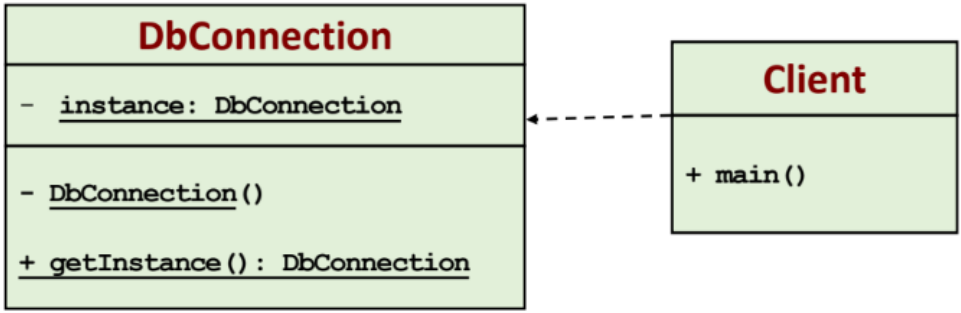
Instead:

- Create one connection object.
- Reuse the same object.
- Provide a global access point.

Goal

Only **one instance** of the class should exist.

Singleton – Class Structure



The constructor is private so that clients cannot freely create instances.

Singleton – Database Connection

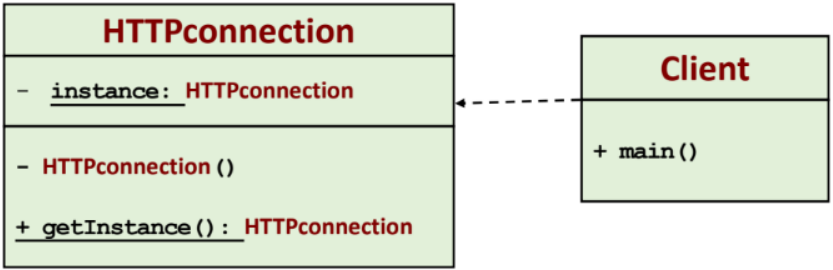
1. Client requests the database connection.
2. Singleton checks whether an instance already exists.
3. If no instance exists, it creates one.
4. The same instance is returned for subsequent requests.

Client $\rightarrow$ `getInstance()` $\rightarrow$ ONE DbConnection

Singleton – HTTP Connection

The same idea can be applied to an HTTP connection.

Example: HTTP Connection



Design Patterns – Summary

Pattern	Main Purpose
Factory Method	Object creation through a factory
Abstract Factory	Factory of related factories/products
Singleton	Ensure only one instance exists

Key Takeaway

Design patterns provide proven, reusable approaches to common software design problems.

Design Patterns – Key Questions

- **Factory Method**
  "Which object should I create?"
- **Abstract Factory**
  "Which family of objects should I create?"
- **Singleton**
  "How can I ensure only one object exists?"

Design once. Reuse the idea.

Thank You

Questions?